Unit 2 · Values and calculations

CONCEPT

Unit 1 used fixed values to demonstrate a sequence of instructions. Unit 2 lets a user supply values while the program runs, then calculates and explains the result.

EXAMPLE / TAKEAWAY

Learning outcomes: recognise common data types; apply arithmetic operators; read and convert keyboard input; format output; test an everyday calculator.

The learning journey

CONCEPT

Understand the problem before writing code. Identify inputs, choose the processing steps, and define clear outputs.

EXAMPLE / TAKEAWAY

Input typed by student → text returned by input() → conversion to numbers → calculation → labelled result.

What is a data type?

CONCEPT

A data type identifies the kind of value being handled. Python distinguishes whole numbers, decimal-capable numbers and strings of characters.

EXAMPLE / TAKEAWAY

int: 4 or 2500

float: 2.5 or 1500.75

str: "Amina" or "2500"

Quotes turn numeric-looking digits into text.

Inspecting a value with type()

CONCEPT

The type() function reports the type of a value. It helps explain why apparently similar values behave differently.

EXAMPLE / PYTHON

```
quantity = 4
price = 2500.50
code = "04"
print(type(quantity))  # int
print(type(price))     # float
print(type(code))      # str
```

Variables carry meaning

CONCEPT

A variable is a name associated with a value. A useful name describes the information, such as fare_per_trip or weekly_total.

EXAMPLE / PYTHON

```
fare = 1500
trips = 6
total = fare * trips
print(total)  # 9000
```

Changing a value is not recalculating

CONCEPT

Assignments replace the value associated with a variable name. A previous calculation remains unchanged until its statement runs again.

EXAMPLE / PYTHON

```
trips = 8
print(total)  # still 9000
total = fare * trips
print(total)  # now 12000
```

The main arithmetic operators

CONCEPT

Python uses + for addition, - for subtraction, * for multiplication and / for division. Ordinary division produces a float.

EXAMPLE / TAKEAWAY

7 + 3 → 10

7 - 3 → 4

7 * 3 → 21

7 / 2 → 3.5

Complete groups, remainders, powers

CONCEPT

Floor division // gives the number of whole groups for positive counts; % gives the remainder. The ** operator raises a number to a power.

EXAMPLE / TAKEAWAY

17 // 5 → 3 full bundles

17 % 5 → 2 books left over

2 ** 3 → 8

Order of operations

CONCEPT

Multiplication happens before addition unless parentheses change the grouping. An arithmetically valid expression can still be wrong for the intended problem.

EXAMPLE / TAKEAWAY

10 + 2 * 3 → 16

(10 + 2) * 3 → 36

(5000 + 2000) * 5 → 35000

Use intermediate calculations

CONCEPT

Naming each important result makes an expression easier to read, check and change later.

EXAMPLE / PYTHON

```
food = 5000
transport = 2000
days = 5
daily = food + transport
weekly = daily * days
print(weekly)  # 35000
```

Reading a value with input()

CONCEPT

The input() function displays a prompt and waits for keyboard input. The answer is always returned as text, even if the user types digits.

EXAMPLE / PYTHON

```
name = input("Your name: ")
print("Welcome,", name)

# If the user types Amina:
# Welcome, Amina
```

Why 12 + 3 can become 123

CONCEPT

If two typed values have not been converted, the + operator joins the two strings rather than calculating a numeric sum.

EXAMPLE / PYTHON

```
first = input("First: ")   # type 12
second = input("Second: ") # type 3
print(first + second)       # 123
```

Convert text to a whole number

CONCEPT

Use int() when the answer should be a whole-number count. A number written as text becomes an integer that can take part in arithmetic.

EXAMPLE / PYTHON

```
books = int(input("Number of books: "))
price = 2000
print(books * price)

# Input 3 → output 6000
```

Convert text to a decimal number

CONCEPT

Use float() for a value that may contain a decimal point. The input should contain ordinary numeric digits, without commas or units.

EXAMPLE / PYTHON

```
fare = float(input("Fare (TZS): "))
trips = int(input("Trips: "))
print(fare * trips)

# Input 1500.50 and 2 → 3001.0
```

Typing digits: three different results

CONCEPT

A type affects what the + operator means. String + string joins; number + number adds; string + number raises an error.

EXAMPLE / TAKEAWAY

"12" + "3" → "123"

12 + 3 → 15

"12" + 3 → TypeError

A complete transport calculator

CONCEPT

The program asks two questions, converts text, multiplies the resulting numbers and displays the calculated cost.

EXAMPLE / PYTHON

```
fare = float(input("Fare in TZS: "))
trips = int(input("Trips: "))
total = fare * trips
print(f"Transport cost: TZS
{total:.2f}")
```

Trace the calculator

CONCEPT

For input 1500 and 6: first the typed values are strings; then conversion creates float 1500.0 and integer 6; finally multiplication produces 9000.0.

EXAMPLE / TAKEAWAY

Input → conversion → processing → output

Displayed result:

Transport cost: TZS 9000.00

Strings describe the result

CONCEPT

Strings carry names, labels and sentences. Joining strings with + and displaying values with print() can communicate the meaning of a result.

EXAMPLE / PYTHON

```
name = "Amina"
print("Hello, " + name)
print("Hello,", name)
```

Readable sentences with f-strings

CONCEPT

An f-string includes values or expressions in curly braces. Python evaluates them when the string is created.

EXAMPLE / PYTHON

```
items = 3
price = 2500
print(f"{items} books cost TZS {items * price}")
# 3 books cost TZS 7500
```

Formatting decimal places

CONCEPT

The :.2f format specification displays two digits after the decimal point. Formatting affects the displayed string, not the stored value.

EXAMPLE / PYTHON

```
distance = 12.75
days = 4
average = distance / days
print(f"{average:.2f} km")
# 3.19 km
```

Weekly expenditure: the problem

CONCEPT

Estimate a student’s weekly cost using daily food, daily transport and the number of campus days. Compare the total with the weekly allowance.

EXAMPLE / TAKEAWAY

Inputs: food/day, transport/day, days, allowance

Weekly cost = (food + transport) × days

Balance = allowance − weekly cost

Weekly expenditure: implementation

CONCEPT

Convert typed values, calculate a daily total, multiply by the number of days, then subtract the weekly cost from the allowance.

EXAMPLE / PYTHON

```
food = float(input("Food/day: "))
travel = float(input("Travel/day: "))
days = int(input("Days: "))
allowance = float(input("Allowance: "))
weekly = (food + travel) * days
balance = allowance - weekly
```

Interpreting the balance

CONCEPT

The calculation can produce a positive, zero or negative balance. In Unit 2 it displays the number; Unit 3 will add conditional decisions to interpret the result.

EXAMPLE / TAKEAWAY

Allowance 40000; weekly 32500 → balance 7500

Allowance 20000; weekly 22000 → balance -2000

Common errors and their clues

CONCEPT

A TypeError can come from adding a string to a number. A ValueError can occur when int() receives unsuitable text. Division by zero is also an error.

EXAMPLE / TAKEAWAY

"4" + 2 → TypeError

int("four") → ValueError

10 / 0 → ZeroDivisionError

Read the error message and locate the statement.

Logic errors can look convincing

CONCEPT

A programme may run but calculate the wrong thing. Operator precedence is a common cause, so group daily spending before multiplying by days.

EXAMPLE / TAKEAWAY

Incorrect: food + transport * days

Correct: (food + transport) * days

Use hand calculations to detect logic errors.

Three useful tests

CONCEPT

Test normal input, a zero-day case and a week where the cost exceeds the allowance. Record expected and actual results rather than relying only on a successful run.

EXAMPLE / TAKEAWAY

4000, 2500, 5, 40000 → weekly 32500; balance 7500

3500, 2000, 4, 20000 → weekly 22000; balance -2000

5000, 3000, 0, 10000 → weekly 0; balance 10000

Data types give values meaning

CONCEPT & REASONING

A data type determines which operations a value supports and how it is represented. Python names are not permanently tied to one type: assignment associates a name with an object, and a later assignment can associate it with a different object. int represents whole numbers, float decimal-capable numbers, str text, and bool True or False.

EXAMPLE & RESULT

```
a = 25
b = 25.0
c = "25"
print(type(a), type(b), type(c))
print(a + 5, c + "5")
```

Types: int, float, str. Results: 30 and 255 (text). The same digits can represent a number or a string.

Type conversion and invalid values

CONCEPT & REASONING

input() returns a string, including the spaces and punctuation typed by the user. int() accepts integer-form text; float() accepts decimal-form text. Conversion can fail with ValueError. int(3.9) truncates towards zero; it does not round. Leading zeros in identifiers may be meaningful, so student IDs often remain strings.

EXAMPLE & RESULT

```
print(int("25"))
print(float("3.5"))
print(int(3.9))
# int("3.5") raises ValueError
# int("hello") raises ValueError
```

25, 3.5, 3. An identifier such as "0012" loses its leading zeros when converted to int. Error handling is developed in Unit 6.

Text processing and clean labels

CONCEPT & REASONING

Strings contain characters. len() counts characters; upper() and lower() return new strings in a different case; strip() removes whitespace from both ends. These methods do not change the original string. Concatenation joins strings, while an f-string combines text and expressions.

EXAMPLE & RESULT

```
name = "  Amina  "
clean = name.strip()
print(clean.upper())
print(len(clean))
print(f"Welcome, {clean}!")
```

AMINA; 5; Welcome, Amina! The original name still contains its spaces.

Decimal approximation and division

CONCEPT & REASONING

Floating-point numbers are represented with finite binary precision. Some decimal fractions are approximate, so 0.1 + 0.2 may not equal the stored representation of 0.3. Formatting changes the displayed value, not the stored one. Division by zero raises ZeroDivisionError; integer division // gives the floor, and % gives the remainder.

EXAMPLE & RESULT

```
print(0.1 + 0.2)
print(f"{0.1 + 0.2:.2f}")
print(17 // 5, 17 % 5)
# print(5 / 0) raises an error
```

0.30000000000000004; 0.30; 3 2. Whole shillings can often be stored as integers.

Scientific measurement: temperature conversion

CONCEPT & REASONING

A formula converts a measurement between units. Celsius and Fahrenheit use different scales, so the calculation multiplies by 9/5 and adds 32. The temperature is a float because measurements may include decimals. Clear variable names and units prevent ambiguity.

EXAMPLE & RESULT

```
celsius = float(input("Temperature in C: "))
fahrenheit = celsius * 9 / 5 + 32
print(f"Temperature: {fahrenheit:.1f} F")
```

Input 0 gives 32.0 F; 100 gives 212.0 F; -40 gives -40.0 F. These are independent reference cases.

From measurements to a summary

CONCEPT & REASONING

An average combines several measurements into one representative value. The total is divided by the number of observations, not by their units or by a fixed arbitrary number. Intermediate variables clarify the formula and support testing.

EXAMPLE & RESULT

```
m1 = 12.5
m2 = 14.0
m3 = 15.5
total = m1 + m2 + m3
average = total / 3
print(f"Mean length: {average:.2f} cm")
```

Mean length: 14.00 cm. A programme that silently divides by 2 would run but produce a wrong result.

Practice and independent Activity U2

CONCEPT

Guided exercises explore string conversion, shopping totals and the weekly budget. The independent activity asks for a runnable calculator, an algorithm and test evidence.

EXAMPLE / TAKEAWAY

Submit a .py file and brief notes. Show typed inputs, two labelled TZS outputs and three tests. Explain which conversions are needed.

Before moving to Unit 3

CONCEPT

A learner is ready when they can explain data types, convert text input, use parentheses, format numeric output and check results independently.

EXAMPLE / TAKEAWAY

Unit 3: Making decisions with if, elif and else. Conditions will allow the programme to respond differently to different results.